Manual - Running the Account Enrichment Steps

Living document. We extend this manual as new steps are added. *Last updated: 27 October 2026*

What this pipeline does

Once a year we export the full CRM account list and run it through three small scripts that automatically add two kinds of information:

- **Segment** - what kind of organisation each account is (Local government, Healthcare, IT & software, ...). See step 1b.
- **Population** (`cx_population`) - the number of inhabitants, for the government bodies among the accounts. See step 2.

The result is one Excel file (`final_enriched.xlsx`) that can be loaded back into Dynamics 365. The rest of this manual is the step-by-step run guide; this opening section explains **which accounts get a population number and where that number comes from**.

Which accounts get a population, and from where

A population is only filled for organisations that actually have inhabitants (a municipality does; an IT company does not). The classifier (step 1) labels every account with a type; step 2 then uses the source below per type. If a lookup finds nothing, the account keeps its existing CRM value (it is never blanked).

Account type (example)	Country	Population comes from
Gemeente / municipality (Gemeente Amsterdam)	NL	CBS "Gebieden in Nederland" - the official statistics office, current-year figure. Merged/abolished gemeenten fall back to their last-known figure from Wikidata.
Gemeente / municipality (Gemeente Gavere)	BE	Wikidata (Belgian municipality list). Merged ones (e.g. Zwijndrecht, 2025) fall back to their last-known figure.
Gemeinde / municipality (Stadt Lübeck)	DE	Wikidata, looked up via the official municipality key (AGS). Same-name towns are told apart by the account's postal code.
OCMW (OCMW Gavere)	BE	Population of the **gemeente** it belongs to (same source as BE gemeente).
AGB - autonomous municipal company (AGB Mortsel)	BE	Population of the **gemeente** it belongs to.
Landratsamt / Kreisverwaltung (Landratsamt Kusel)	DE	Population of the **Landkreis** it belongs to.
Provincie / province (Provincie Utrecht)	NL	Wikidata (NL province list).
Provincie / province (Provincie Antwerpen)	BE	Wikidata (BE province list).
Landkreis / district (Landkreis Bautzen)	DE	Wikidata, looked up via the official district key.
Politiezone / police zone (Politiezone Antwerpen)	BE	Sum of the populations of the gemeenten the zone covers (173 zones mapped from the official list).
Veiligheidsregio / safety region (Veiligheidsregio Zeeland)	NL	Sum of its member gemeenten; the membership list comes from CBS (all 25 regions).

Account type (example)	Country	Population comes from
Omgevingsdienst / environmental agency	NL	Sum of its member gemeenten (mapping partially filled).
Waterschap / water authority (Waterschap Aa en Maas)	NL	Fixed table of the 21 water authorities, taken from their own websites.
Stadsdeel / city district (Stadsdeel Centrum)	NL	Fixed table of the 8 Amsterdam districts, from Wikipedia.
Verbandsgemeinde (Verbandsgemeinde Daun)	DE	Fixed table from the German Wikipedia (official Landesamt figures).
Land / country government (Caribbean) (Bestuur Aruba)	AW/CW/SX/BQ	Wikidata country total (Aruba, Curaçao, Sint Maarten, Caribisch Nederland).

Accounts that deliberately get NO population (they have no inhabitants): commercial companies, ministries / Rijksoverheid, Belgian FOD, Stadtwerke, intercommunales, CAW, Zweckverband. These keep an empty population field.

Types not yet automated (they keep their existing CRM value until we build a mapping for them): inter-municipal cooperations (`samenwerking` , `belastingsamenwerking`), GGD health regions, `stadsregio` , BE `hulpverleningszone` , German `Amt` and `Verwaltungsgemeinschaft` . These are relatively few accounts; see the maintenance section for how to add them.

> *Every output row also has a `bron` column naming the exact source used, and a `proces` column explaining the decision - so any single value can be traced back. The technical view of these sources (which code table, how to refresh) is in *"How the population values are sourced"* further down.*

Getting the input from the data warehouse (step 0)

1. Log into Microsoft SQL Server Management Studio (DWH).
2. Connect to the DWH database.
3. Go to Views -> `INT.Account` .
4. Run:

   ```
   SELECT *
   FROM [INT].[Account]
   WHERE statecode = 0
   ```

   The `statecode = 0` filter keeps only **active** accounts; disabled accounts are excluded from the pipeline.
5. Right-click the results grid, choose **"Save Results As..."** and save as `full_input.csv` .
6. Copy `full_input.csv` to the project folder on the Linux server (`~/get-populations/` , see one-time setup below).

The scripts read this CSV directly (UTF-8 with BOM, `NULL` text values are handled). If disabled records accidentally end up in the export, step 1 logs a warning.

The pipeline at a glance

The scripts run as a chain. **Order matters**: each step's output is the next step's input, so all added columns accumulate into one final file.

```
full_input.csv (DWH export, see step 0)
  |
  |  python step1_classify.py --input full_input.csv --output step1_classified.xlsx
  v
step1_classified.xlsx          (+ type/country detection columns)
  |
  |  python step1b_segment.py --input step1_classified.xlsx --output step1b_segmented.xlsx --web-search
  v
step1b_segmented.xlsx          (+ Segment, Segment (detailed) columns)
```

```
  |
  |  python step2_enrich.py --input step1b_segmented.xlsx --output final_enriched.xlsx \
  |      --user-agent "your-org/1.0 (you@company.com)"
  v
final_enriched.xlsx            (+ population columns) <- THE final file
```

Step 1 is free and instant. Step 1b uses the Claude API (costs money, uses the cache). Step 2 uses Wikidata (free; the first run downloads reference data, which takes 5-10 minutes, after that it is cached in `reference/`).

The separate **TAM tool** (`tam_build.py` , section 7) starts from `step1b_segmented.xlsx` and builds the white space list; it does not feed step 2.

Every run of every step also writes a timestamped log file to `logs/` (e.g. `logs/step1b_segment_2026-06-11_143052.log`), so a failed or unattended run can be diagnosed afterwards. The `logs/` folder stays out of git.

Which steps need the paid Claude API?

Step	Needs
step 0 (DWH export)	Nothing (internal DWH access)
step 1 (classify)	Nothing - runs locally, free
step 1b (segment)	**Anthropic API key + prepaid credits**
step 2 (population)	Nothing - Wikidata is free (just put a real contact email in `--user-agent`)

Important: a Claude.ai subscription (Pro or Max) does **not** cover API usage. The API is billed separately via prepaid credits, even if you already pay for a Claude subscription. Claude Code / claude.ai usage and API usage are two different wallets.

Getting an API key and credits (one-time)

1. Go to console.anthropic.com and sign in (or create an account).
2. Under **Billing / Plans & billing**, buy prepaid credits with a credit card. $50 comfortably covers the first full run plus re-runs; yearly refreshes after that cost a few dollars at most.
3. Under **API Keys**, create a new key and copy the `sk-ant-...` value (it is shown only once).
4. Store it on the server:

   ```
   echo 'export ANTHROPIC_API_KEY="sk-ant-..."' >> ~/.bashrc
   ```

0. One-time setup (first time on a machine)

This assumes a fresh Linux box with `git` and `python3` installed and nothing of this project yet. We install everything under the home directory (`~/get-populations`); no root access is needed.

```
# 1. get the code from GitHub
cd ~
git clone https://github.com/Golden-VS/get-populations.git
cd get-populations

# 2. create a Python environment and install all dependencies
python3 -m venv venv
source venv/bin/activate
pip install pandas openpyxl anthropic requests rapidfuzz
```

Set the Anthropic API key (from console.anthropic.com, where the credits live - only step 1b needs it):

```
export ANTHROPIC_API_KEY="sk-ant-..."
```

To avoid retyping it every session, store it once in your shell profile:

```
echo 'export ANTHROPIC_API_KEY="sk-ant-..."' >> ~/.bashrc
```

Before every working session, go to the folder and activate the environment first:

```
cd ~/get-populations
source venv/bin/activate
```

1. Classification (step 1)

```
python step1_classify.py --input full_input.csv --output step1_classified.xlsx
```

- Free and fast (a few seconds for ~6,000 records); no API needed.
- Detects per account the entity type (gemeente, waterschap, politiezone, commercial, ...) and the country, based on the account name.
- Check the summary in the terminal (also in the `logs/` file): - the distribution per `detected_type` should look plausible (hundreds of municipalities, no unexpectedly empty types); - the *"'Onbekend' maar wel een cx_businesstype"* list and the *conflicts* table show records worth a quick manual glance.
- Output `step1_classified.xlsx` is the input for step 1b.

2. Segmentation test run (step 1b, 100 accounts)

Always do this first after any change to the script or the input file.

```
python step1b_segment.py \
    --input step1_classified.xlsx \
    --output step1b_test.xlsx \
    --test-mode --web-search
```

- Takes a few minutes, costs well under $1.
- Open `step1b_test.xlsx` and check the columns "Segment", "Segment (detailed)", "Segment (category)" (Governmental / Non-profit / Commercial / Unknown), `segment_confidence` and `segment_bron` .
- Pay extra attention to rows with confidence `low` and segment `Commercial (other)` or `Unknown` .

3. Segmentation full run (step 1b, all ~6,000 accounts)

```
python step1b_segment.py \
    --input step1_classified.xlsx \
    --output step1b_segmented.xlsx \
    --web-search
```

- Takes roughly 1 to 1.5 hours. Progress is logged per batch.
- Estimated cost: $15 to $25 including web searches (covered by the $50 credit purchase).
- Safe to interrupt: results are saved to `segment_cache.csv` after every batch. Re-running the same command resumes where it stopped and does not pay twice for accounts already classified.
- At the end the log prints a count per segment. Sanity-check that distribution (e.g. roughly 1,500 municipalities, no giant "Unknown" bucket).

4. Corrections

If a segment is wrong, do not edit the output Excel by hand (it would be overwritten on the next run). Instead, add a row to a corrections file, for example `overrides_segment.xlsx` , with these columns:

accountid	segment_override	segment_detailed_override	reden
(CRM guid)	IT & software	Software vendor	manually verified

Then re-run with:

```
python step1b_segment.py \
    --input step1_classified.xlsx \
    --output step1b_segmented.xlsx \
    --web-search --overrides overrides_segment.xlsx
```

Overrides always win over the automatic classification and cost nothing.

5. Population enrichment (step 2)

After segmentation, run step 2 on **step1b's output** (not on step1_classified.xlsx, or the Segment columns will be missing from the final file):

```
python step2_enrich.py \
    --input step1b_segmented.xlsx \
    --output final_enriched.xlsx \
    --user-agent "your-org/1.0 (you@company.com)"
```

- Wikidata requires a contact address in `--user-agent` . The whole string is free text in the form `toolname/version (email)` - both parts are yours to choose, nothing is registered anywhere. It only serves to identify the caller so Wikimedia can contact you if a query misbehaves. Concrete example:

```
--user-agent "crm-enrichment/1.0 (jan.jansen@yourcompany.nl)"
```

Pick any sensible tool name, keep `1.0` as version, and use a real work email.

- First run downloads reference data (5-10 min); later runs use the cache in `reference/` (refreshed yearly automatically, or force with `--refresh-cache`).
- Use `--test-mode` here too when running for the first time.
- `final_enriched.xlsx` is the complete file: original columns + type detection + segments + population.

How the population columns work

- `cx_population` is updated *in place* (same column name as in the CRM, so the file can be imported back directly). If no new value is found for a record, the old CRM value is **kept**, never blanked.
- `previous_population` (new column) holds the old CRM value as it was before this run, so you can filter on records where the value changed.
- `population_gewijzigd` (new column): `gewijzigd` / `gelijk` / `leeg` . Filter on `gewijzigd` to review exactly which values would change in the CRM; the `run_log` tab also shows the total count.
- Supporting columns explain every value: `bron` (source, incl. URL or Wikidata ID), `proces` (short explanation in Dutch), `peildatum_inwoners` (reference year of the statistic), `data_leeftijd_jaren` (age of that statistic in years - high values flag stale data), `match_score` and `invuldatum` .

Population corrections

Step 2 has its own corrections table (separate from the segment one). Create e.g. `overrides_population.xlsx` with columns:

account_id	population_override	reden
(CRM guid)	154000	figure from annual report 2025

and run with `--overrides overrides_population.xlsx` . Overrides win over every automatic lookup.

How the population values are sourced (technical / maintenance view)

The table at the very top of this manual lists the source **per account type**. This section is the maintenance view: the four underlying **mechanisms** behind those sources and **where each lives in the code**, so you know what to touch when something needs updating.

Mechanism	How it works	Where in the code
1. Wikidata lookup	Downloads reference lists with populations from Wikidata (free, no key) and fuzzy-matches account names. Same-name German municipalities are told apart by postal code; dissolved NL/BE gemeenten fall back to a frozen last-known figure via an exact-name list.	`REFERENCE_SOURCES` in `step2_enrich.py` ; cached in `reference/*.csv`
2. CBS open data	The official "Gebieden in Nederland" table - current-year NL gemeente figures, and the veiligheidsregio membership used by mechanism 3. Auto-discovers the newest yearly edition.	`fetch_cbs_gebieden()` ; cached in `reference/cbs_gebieden.csv`
3. Sum of member gemeenten	Entity population = sum of the gemeenten it covers (politiezones, veiligheidsregio's, omgevingsdiensten).	`BE_POLITIEZONE_GEMEENTEN` , `NL_OMGEVINGSDIENST_GEMEENTEN` ; veiligheidsregio mapping from CBS at run time (inline table is the fallback)
4. Direct-value tables	Hand-collected numbers with a source URL per entry, where no database has the data (waterschappen, Amsterdam stadsdelen, DE Verbandsgemeinden).	`NL_WATERSCHAP_INWONERS` , `NL_STADSDEEL_INWONERS` , `DE_VERBANDSGEMEINDE_INWONERS`
5. Manual override	A corrections file that wins over everything.	`--overrides` file (see above)

If none of these produce a value, the old CRM value is kept (never blanked). Every output row's `bron` column says exactly which source produced its value.

When a source fails or looks wrong (yearly maintenance)

Start with the **"Status per bron"** report at the end of the step 2 log: every reference source is listed as OK / leeg (empty) / GEFAALD (failed).

A Wikidata source is empty or failed:

1. Delete that source's file from `reference/` and re-run (forces a fresh download; transient server errors are retried automatically).
2. Still broken? The Wikidata class may have changed. Test the query on https://query.wikidata.org/ and check `CHECKPOINT.md` , which documents how every Q-ID/key was found and validated. Expected healthy counts: NL gemeenten ~1,300 (incl. historical), BE gemeenten ~565, NL provincies 12, BE provincies 10, DE gemeinden ~11,400, DE landkreise ~290.

The CBS source fails: the run does not stop - it logs a warning ("cbs_gebieden GEFAALD") and falls back to Wikidata population figures and the built-in veiligheidsregio table. Values are then 1-2 years older but still correct. Remedy: delete `reference/cbs_gebieden.csv` and re-run; if CBS renamed the table series, check `discover_cbs_gebieden_table()` in `step2_enrich.py` .

A count is suddenly way off (e.g. BE gemeenten drops to 50): treat as broken even if the run "succeeds" - the fuzzy matcher will quietly miss records. Same remedy as above.

Direct-value tables age (methods 3): the numbers drift ~1%/year. Refresh by checking each entry's `bron_url` ; for waterschappen there is a built-in sanity check: the 21 values must sum to roughly the NL population (~18 million). A record that says *"niet in ...-tabel"* in its `proces` column means a new entity appeared (e.g. a new Verbandsgemeinde in the CRM): add one line to the relevant table with the figure from the source in the comment above that table.

Mapping tables outdated after a municipal reorganisation (method 2): politiezone mergers happen in BE every year. The `BE_POLITIEZONE_GEMEENTEN` table was extracted from the NL Wikipedia page "Lijst van politiezones in België"; the extraction recipe is documented in `CHECKPOINT.md` and can be re-run.

General health checks after any run:

- `proces` column: filter on "geen match" / "niet in" to see what was missed.
- `proces` column: filter on "LET OP" to find name collisions that could not be resolved (e.g. a German municipality name that exists in several states while the account has no postal code) - verify those by hand.
- `data_leeftijd_jaren` column: high values (5+) flag stale source data.
- `previous_population` vs `cx_population` : large jumps deserve a look.

6. Next year's run (yearly refresh)

1. Make a fresh DWH export following **step 0** at the top of this manual and place `full_input.csv` in the project folder.
2. Run the full chain from the pipeline diagram: step 1 -> step 1b (with `--overrides ...` if you have a corrections file) -> step 2.
3. Thanks to `segment_cache.csv` , only **new accounts and accounts whose name changed** are sent to the AI. Everything else comes from the cache. Expected cost: a few dollars at most, usually cents.
4. Step 2's `reference/` cache is older than 365 days by then, so it re-downloads fresh Wikidata population data automatically.

Do **not** delete `segment_cache.csv` ; it is the memory of all previous classifications. If you ever want to force a complete re-classification (for example after a major change to the segment list), run once with `--refresh-segments` and expect full-run costs again.

7. TAM / whitespace list (`tam_build.py`)

What it does

Step 2 answers "how many inhabitants does this CRM account serve?". The TAM tool answers the opposite question: **which governmental organisations exist at all** in NL, BE and DE, how large are they, and which of them are already customers? It builds a master list of all relevant bodies (municipalities, provinces, states, districts, water authorities, safety regions, police zones, inter-municipal cooperations and so on) from official registers, matches the CRM accounts to that list, and writes one Excel file that Power BI uses to compute white space (potential not yet served) and black space (already served).

It runs about once a year. The master list comes from registers (not from the CRM), so it also contains organisations that are not customers.

Prerequisites

1. **A fresh CRM export**, processed through step 1 and step 1b (sections 1 to 3), so `step1b_segmented.xlsx` covers all current accounts.
2. **The API key** in a file called `.env` in the repo root (gitignored, permissions 600) with one line `ANTHROPIC_API_KEY=...` . `tam_build.py` reads this file itself. Step 1b does not, so before step 1b run `set -a; . ./.env; set +a` once in your shell.
3. `tam_config.json` (gitignored). Copy `tam_config.example.json` and fill in your own values: - `companies` : the allow-list of companies whose products are in scope; - `ignore_core_products` : product families that never get a TAM row.
4. **The product mapping workbook** (first tab) in `doc/` , or pass another file with `--products` .

Step A: make the template

```
venv/bin/python tam_build.py --make-template
```

This writes `tam_template.xlsx` with sheet `tam_definitions` : one row per core product and country (NL, BE, DE). A core product that appears under several sales units is assigned to the sales unit with the most product lines; the strays are listed in the run log. The command refuses to overwrite an existing template. Next year, use `--make-template --update` : it only appends rows for new core products and leaves everything already filled in untouched.

Step B: the business fills `tam_definitions`

The first four columns (`company` , `sales_unit` , `core_product` , `country`) are filled in. The business fills in:

Column	Rule
`segments`	One or more top-level segments for this product and country, separated by `;` : `Local government` , `Regional government` , `National government` , `Inter-municipal cooperation` , `Public safety & emergency` (for example `Local government;Inter-municipal cooperation`). Allowed values are in the first column of sheet `lists` . **Rows with empty `segments` are skipped.**
`excluded_kinds`	Optional. Detailed segments (second column of sheet `lists`) to leave out again, separated by `;` (for example `City district;Municipal autonomous agency`). This narrows the top-level segments when a product fits a whole group except a few kinds.
`min_population`	Optional whole number. Entities below it are marked `no - below cutoff` . Empty means no cut-off.
`notes`	Free text, copied to the output (last column of sheet `TAM`).

An unknown value stops the run with a message that names the row and the value, so typos are caught before anything costs money. Two filled rows for the same company, core product and country also stop the run (they would count the TAM twice). The segment values are a fixed list (also the join key for the factor table in Power BI), so do not rename them between years.

Sheet `match_overrides` is described under "Reviewing the matches" below.

Step C: dry run, then the real run

```
venv/bin/python tam_build.py --dry-run
```

The dry run downloads the registers (cached in `reference/`), builds the master list, does the free rule-based matching and prints the **estimated AI cost** against the cap. It makes no AI call and writes no output file. It needs no API key. Check the estimate, then run for real. In the yearly run, force fresh register downloads:

```
venv/bin/python tam_build.py --refresh-cache
```

Useful options:

Option	What it does
`--config <file>`	Other config file (default `tam_config.json`)
`--template <file>`	Other template file (default `tam_template.xlsx`)
`--crm <file>`	Other CRM file (default `step1b_segmented.xlsx`)
`--output <file>`	Other output name (default `tam_<date>.xlsx`)
`--refresh-cache`	Download all registers again (use in the yearly run)
`--offline`	Only use cached downloads
`--dry-run`	Everything up to the AI estimate; no AI call, no output
`--no-ai`	No AI at all: cooperations keep a generic segment and are flagged, name matching pass 2 is skipped
`--max-cost <usd>`	Cost cap for this run (default 50)

Cost cap. The tool prints its estimate before any AI call and tracks the real spend during the run. Before each AI call it checks the worst case of that call (the prompt plus the longest possible answer), so the cap is never passed. If the cap would be exceeded it stops cleanly, the caches are saved, and a re-run resumes where it stopped. Governmental accounts are processed first, so a cap never starves them. Because both AI steps cache their

answers (cooperation classification per entity, matches per account), a repeat run costs very little.

Cooperation classification. Cooperations get their detailed segment from the AI, based on name, country, kind, member count and, for Dutch cooperations, the purpose text ("doel") from the government register. A cooperation is classified again only when its name or purpose text changes.

Register downloads that fail. If a register cannot be downloaded but an older copy is in `reference/`, the tool uses that copy, logs a warning and marks the source in the run log with "verouderde cache, datum ..." (the date of the cached file). Only a source without any cached copy fails.

Step D: reading the output

The result is `tam_<date>.xlsx` (gitignored) with these sheets:

Sheet	Contents	Use in Power BI
`TAM`	One row per (template row x entity in the selected segments): company, sales unit, core product, country, `entity_id`, name, `detailed_segment`, `population`, `in_tam`, `in_tam_reason`, number of matched accounts, `notes` from the template	The white space base
`CRM matches`	One row per matched account: `entity_id`, `accountid`, CRM name, `match_pass`, `match_reason`, `confidence`	Join revenue per `accountid` to `entity_id`, then to `TAM` (black space)
`Entities`	The full master list with all attributes (kind, region, ids, population method, source and date)	Audit, optional dimension table
`Memberships`	Cooperation `entity_id` -> member `entity_id`, with the cooperation's `detailed_segment` and join date	Members of a paying cooperation count as covered
`CRM unmatched`	Governmental or unknown CRM accounts in NL/BE/DE without an entity, with best candidate and why it was rejected. Accounts abroad that step 1b marked Governmental appear with reason "land buiten scope"	Spot paying customers that are missing in the TAM
`Run log`	Inputs (including the modification date of the CRM file), per-source status and dates, counts per country and kind, counts per `in_tam` status, population-unknown counts, AI calls and cache hits, product strays, new segment values, open points, and warnings (including source-level warnings from the register loaders)	Check after every run

`in_tam` values

Value	Meaning
`yes`	In the TAM for this product
`no - below cutoff`	Population below `min_population`
`no - excluded kind`	Its detailed segment is in `excluded_kinds`
`unknown - population missing`	A cut-off is set but the population is not known

`match_pass` values (sheet `CRM matches`)

Value	Meaning
`rule`	Free, deterministic: exact company number (KvK/KBO) or unique normalised name within a kind compatible with the account's type from step 1. German same-name hits are decided by postcode; when the rule had to choose between kinds (for example a municipality and an Amt that has a member with that name) it only matches if the account's postcode is close to the chosen entity and no farther than the alternatives. Accounts without a recognised governmental type never match on name here; they go to the AI step
`ai`	The AI chose between a shortlist of candidates
`override`	Set by you in `match_overrides`

Memberships and covered members. Only direct cooperation memberships are exported, never the territorial relation province/district/state -> municipality (otherwise one paying province would mark all its municipalities as covered). The sheet carries the cooperation's `detailed_segment` , so Power BI decides which kinds of cooperation make their members covered (for example only shared service and IT cooperations). Cooperations of cooperations are not chained.

Reviewing the matches

1. Open `CRM matches` and filter `confidence = low` (AI matches the model was not sure about). They are kept in the result, so check them.
2. Open `CRM unmatched` and look for customers that should have matched.
3. Fix a wrong or missing match in sheet `match_overrides` of the template (CRM `accountid` and the `entity_id` it belongs to; an empty `entity_id` means "this account is not a TAM entity"), then run again. Overrides win over rules and AI. An override pointing at an unknown `entity_id` is reported as an ERROR in the run log, not silently skipped. An override for an `accountid` that is not in the CRM file is ignored with a warning in the run log.

Yearly refresh

1. Fresh CRM export (step 0), then step 1 and step 1b (the segment cache keeps this cheap).
2. `venv/bin/python tam_build.py --make-template --update` , and let the business check the new rows.
3. `--dry-run` , check the estimate against the cap.
4. Real run with `--refresh-cache` , then review `Run log` , low-confidence matches and `CRM unmatched` .

Notes for the Power BI model

- **White space value** = `population x municipal ARPU per inhabitant x factor per detailed segment` (for example a province pays roughly 10% of the municipal rate per inhabitant). The business defines and maintains the factor table, keyed on `detailed_segment` .
- Rows with `in_tam = unknown - population missing` have no euro value; Power BI decides how to show them.
- Populations are not deduplicated between levels: a province and its municipalities both carry their population. The per-kind factor takes care of this.
- Black space uses recurring revenue only (`Billing Type = Recurring`).
- **Phase 2, winnable per year**: `winnable per year = white space value x churn rate` . With 10% churn, about 10% of the white space comes up for grabs each year (assumption: competitors churn like we do). This ignores that the winnable share shrinks as market share grows. Use the churn rate of the same core product (and country if available). Computed in Power BI; the tool needs nothing extra.

Known limitations

These are visible in the `Run log` ; none stops the run.

- **Belgian police zones**: 552 of 565 municipalities are covered by a police zone (the German-speaking community and a few others are missing in the source table).
- **Population unknown**: Aruba and the Walloon-Brussels Federation, Walloon and Brussels intercommunales (no member lists), most German Zweckverbaende and 28 of 30 IT Zweckverbaende. They appear with `population_method = unknown` .
- **Two member lists cite secondary sources**: the members of the KVJS (Baden-Wuerttemberg) and of the Bezirksverband Pfalz are the sets fixed by law (all Stadt- and Landkreise of the Land; the Kreise of the former Regierungsbezirk Pfalz), cited from a legal text mirror and a state civic education publication because the official law portals could not be read as text.
- **Licence of the Flemish organisation portal dump** (Belgian police zones, AGBs, cooperations) is not stated in its metadata; the run log lists this as an open point.
- **28 Saxon Verwaltungsgemeinschaften** could not be tied to their executing municipality and are modelled as Verband entities, so they are segmented as Municipal federation.
- **Dutch cooperations that list dissolved municipalities** (old member codes after a merger, for example a youth care cooperation in the Rotterdam area) understate their population; old codes are not remapped.
- **Predecessor aliases** (CRM accounts still named after a merged municipality) come from Wikidata and, for Flanders, from the 2019 and 2025 mergers in the Flemish organisation portal. They are sparse for Wallonia and DE; unmatched ones show up in `CRM unmatched` .
- **No KBO register yet**: Belgian matching on company number only works where the Flemish portal carries it.
- A KvK/KBO number shared by several entities is never used for a rule match; those accounts go to name matching.

Useful options (step 1b)

Option	What it does
`--test-mode`	Only the first 100 accounts
`--web-search`	Second pass with web lookup for weak classifications (recommended). Only searches records that were never websearched before, so re-runs stay cheap
`--offline`	No API calls: mapping table + cache only
`--model <id>`	Use a different Claude model (default `claude-opus-4-7`)
`--overrides <file>`	Apply a manual corrections file
`--refresh-segments`	Ignore the cache, classify everything again

Future steps (to be added to this manual)

- Importing the result columns into Dynamics 365
- Upgrading step 2 to the official registers used by the TAM tool